



REPUBLICA BOLIVARIANA DE VENEZUELA
MINISTERIO DEL PODER POPULAR PARA LA EDUCACIÓN
UNIVERSITARIA
UNIVERSIDAD POLITECNICA TERRITORIAL DE CARACAS
"MARISCAL SUCRE"
INGENIERIA INFORMATICA
SECCION 7122

Proyecto 3: Programación orientada a objetos Sistema Avanzado de Gestión de Personal y Proyectos

Profesor: José Gregorio

Autores:

Jhoan Meza CI: 18.329.471

Caracas, 25 de noviembre del 2025

Informe final

(sistema avanzado de gestión de personal y proyectos)

Este sistema esta orientado a una empresa de tecnología que requiere gestionar su personal y los proyectos en los que participan.

Para tal fin se desarrollará un programa que tenga 3 clases en donde se implementará la estructura de cada clase según los datos requeridos por la misma y la forma de relación entre ellas para que funcione correctamente, estas clases son:

1. Clase base abstracta empleado.
2. Clases base concretas derivadas.
 - Desarrollador
 - Gerente
 - Diseñador
3. Clase proyecto (composición)

A continuación, mostramos la estructura que debe tener cada clase según lo indica la programación orientada a objetos (POO).

Ejemplo: Nombre de la clase (ejemplo: Empleado).
 Atributos (ejemplo: nombre: str, id_empleado: int).
 Métodos (ejemplo: calcular_salario (): float).

Ahora definiremos los atributos y métodos utilizados para cada clase.

1. Clase base abstracta empleado.

Atributos privados:
• nombre(str),
• Id_empleado (int)
• Salario base(float)

2. Clases base concretas derivadas.

desarrollador
• lenguajes (lista de string)
• nivel ("junior", "semisenior", "senior (int)
• salario base(float)

2. Clases base concretas derivadas.

gerente
• departamento (string)
• equipo (lista de empleados a su cargo)
• salario base(float)

2. Clases base concretas derivadas.

diseñador
• Herramientas (lista string)
• Especialidad ("UI", "UX", "Grafico")
• salario (float)

3. Clases proyecto.

diseñador
• nombre (string)
• presupuesto ("float")
• empleados (empleados asignados)
Métodos
• agregar_empleado(Empleado)
• costo_total ()
• viabilidad ()

4. Requisitos adicionales.

Los requisitos de herencia, composición, encapsulamiento y poliformismo se usaron de la siguiente manera.

- Herencia: Se implementó mediante la clase empleado.
- Composición: Se utilizó al incluir una instancia de la clase Dirección dentro de persona.
- Encapsulamiento: Se aplicaron en atributos privados como `__salario` y métodos públicos `get_salario ()` y `set_salario(valor)` para controlar el acceso y modificación de datos sensibles.
- Poliformismo: se usó en el método `calcular_pago()` redefinido en las clases Empleado y Freelancer.

5. Por último se realizó una prueba del sistema

```
== Viabilidad de proyectos ==  
Paginas Web: costo= $7600.00 | Viable = True  
Movilnet: costo= $7600.00 | Viable = True  
Leonor alcanzo limite de 3 proyectos
```

DIAGRAMA DE CLASES.

